

# AmazonHelp AI Support Agent

Hiver SDE Intern take-home. Goal: turn messy Twitter support data into a trustworthy AI agent for one brand, then prove it with evaluation. Brand: @AmazonHelp. Dataset: Kaggle thoughtvector/customer-support-on-twitter. LLM: Groq openai/gpt-oss-120b (free tier). Embeddings: local all-MiniLM-L6-v2 via ChromaDB (no paid embedding API).

## 1. End-to-end project walkthrough

Step A - Data: `download_data.py` pulls the Kaggle CSV; `data_processing.py` keeps only AmazonHelp replies linked to the customer tweet they answered, producing `customer_text` / `brand_text` pairs plus a keyword heuristic `intent` for exploration.

Step B - Retrieval: `rag_pipeline.py` indexes ~3000 customer questions into a local ChromaDB collection and stores the historical AmazonHelp reply as metadata.

Step C - Agent (`agent.py`): for each new message, retrieve top-3 similar past issues, then call Groq to return structured JSON: `intent`, `draft_reply`, `action` (auto-handle|escalate), and `reason`. Drafts are constrained to Twitter length and grounded in retrieved brand tone (often DM / ask for order details).

Step D - Golden set: `golden_set_generator.py` samples ~200 intent-balanced examples, proposes silver labels with Groq, and leaves `human_intent` / `human_action` / `reviewed` columns for mandatory manual verification (see `docs/GOLDEN_SET_METHODOLOGY.md`).

Step E - Evaluation harness: `evaluation.py` compares (1) trivial baseline, (2) zero-shot Groq without RAG, (3) full RAG agent on intent/action accuracy; runs an LLM-as-judge rubric on drafts; optionally measures judge vs human agreement from `data/human_reply_ratings.csv`; logs everything to MLflow.

## 2. Problem framing

What 'good' means for AmazonHelp: correct intent; a concise, empathetic reply that mirrors how Amazon historically resolves similar tweets; escalate when money, account access, or high emotion appears; auto-handle only clear FAQs / status asks.

What we chose not to build: multi-turn conversation state, live order-database lookups, fine-tuned classifiers, Banking77 intent transfer, and production ticket routing. Proof of trustworthiness matters more than product surface area.

## 3. Results vs baselines

Trivial: majority intent + always escalate. Simple: zero-shot Groq, no RAG. Agent RAG: Groq + top-3 historical pairs. Metrics from `evaluation.py` on n=30 golden examples (LLM judge n=18; judge vs human overlap n=17):

- Trivial - intent acc: 0.40 | action acc: 0.57
- Simple (zero-shot) - intent acc: 0.87 | action acc: 0.77
- Agent RAG - intent acc: 0.90 | action acc: 0.67
- LLM judge means - overall: 4.39, groundedness: 4.11, helpfulness: 4.39, tone: 4.83, safety: 4.94
- Judge vs human - exact: 0.47, within+/-1: 1.00, Cohen kappa: 0.07 (n=17)

## 4. Failure analysis (top 5 with hypotheses)

- Ambiguous mentions ('@AmazonHelp' only): unactionable; agent may invent a helpful tone. Hypothesis: missing parent-thread context.
- Sarcasm ('Great job losing my package'): polarity flips. Hypothesis: short tweets lack affective cues for reliable detection.
- Multi-intent (billing + shipping): single-label taxonomy forces one primary class. Hypothesis: mutually exclusive intents by design.
- Noisy RAG neighbors: generic historical replies pollute drafts. Hypothesis: embedding similarity does not equal resolution quality.
- Escalation over-triggering on refund keywords: FAQs still escalate. Hypothesis: no order-state API; policy prefers false escalations over silent auto-refunds.

## 5. What is misleading about my headline number?

Intent accuracy on LLM silver labels overstates quality until humans finish review. The LLM-as-judge may share style preferences with the drafter (same Groq family), inflating reply scores. Coarse 5-class intents hide multi-intent errors. Free-tier defaults (EVAL\_SAMPLE\_SIZE=30) make headline accuracy noisier than a full 200-example run. Always read accuracy together with judge scores and failure cases.

## 6. What I'd do with one more week

Finish human review of all 200 labels; expand human reply ratings to 50+; add a second independent judge model; restore parent-tweet context; train a cheap local intent classifier to save free-tier quota; ship a Streamlit demo for live reviews.

## 7. Decision log (15 non-obvious choices)

- Brand = AmazonHelp: dense transactional reply patterns suit RAG grounding.
- Switched Gemini -> Groq openai/gpt-oss-120b for free, reproducible inference.
- OpenAI-compatible JSON object mode for portable structured outputs.
- Local MiniLM embeddings via Chroma default: zero embedding API cost/limits.
- Client throttle (~2.2s) to stay under ~30 RPM free limits.
- Default eval n=30 / judge n=20; overridable via env for fuller runs.
- Silver labels then human review to reach 150-250 examples practically.
- Intent-balanced sampling to avoid majority-class-only golden sets.
- Escalation policy encoded in prompt (no live account/order API).
- Single-turn Q->A pairs only; full Twitter graphs out of MVP scope.
- Five-dimension LLM-as-judge rubric for reply quality deliverable.
- Explicit human\_reply\_ratings.csv contract for judge alignment evidence.
- MLflow for side-by-side trivial / simple / RAG comparison.
- Prefer false-positive escalations over silent auto-handling of money issues.
- Separate methodology doc so reviewers can audit labeling choices.

## 8. Manual setup (free tier) and submission

1) Create Groq key at console.groq.com/keys and set GROQ\_API\_KEY in .env. 2) Install: python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt. 3) Data: python download\_data.py (or unzip twcs.csv into data/). 4) python data\_processing.py && python rag\_pipeline.py. 5) python golden\_set\_generator.py then hand-review labels. 6) Copy data/human\_reply\_ratings.example.csv -> human\_reply\_ratings.csv and score agent drafts. 7) python evaluation.py && python generate\_report\_pdf.py. Submit via Notion form only (repo link + this report). Do not email.